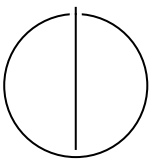
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis, Master's Thesis, ... in Informatics

Thesis title

Author



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

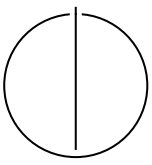
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis, Master's Thesis, ... in Informatics

Thesis title

Titel der Abschlussarbeit

Author:	Author
Examiner:	Supervisor
Supervisor:	Advisor
Submission Date:	Submission date



I confirm that this bachelor's thesis, master's thesis, ... is my own work and I have documented all sources and material used.

Munich, Submission date Author

Acknowledgments

Abstract

For compilers, compiletime and runtime are fundamentally a tradeoff. Therfore frameworks like LLVM provide multiple backends for faster compilation or better codegeneration.

Recently TPDE has shown that compilation speed of LLVM -O0 can be improved by around 20x while keeping a similar runtime. Our work aims to expand TPDE to achieve better codegeneration, while keeping a fast compiletime.

We present a novel register allocation approach, that works without knowledge of the IR instruction semantics and keeps the single-pass codegeneration step introduced by TPDE.

On Spec Int 2017 we achieve a 30% runtime improvement over the LLVM -O0 back-end, while keeping a 3x faster compiletime on optimized IR. Our approach shows the potential for faster lightly optimizing compilers, although we are limited by implementation issues, as well as the LLVM middle end transforamtions leading to difficult to optimize IR for TPDE.

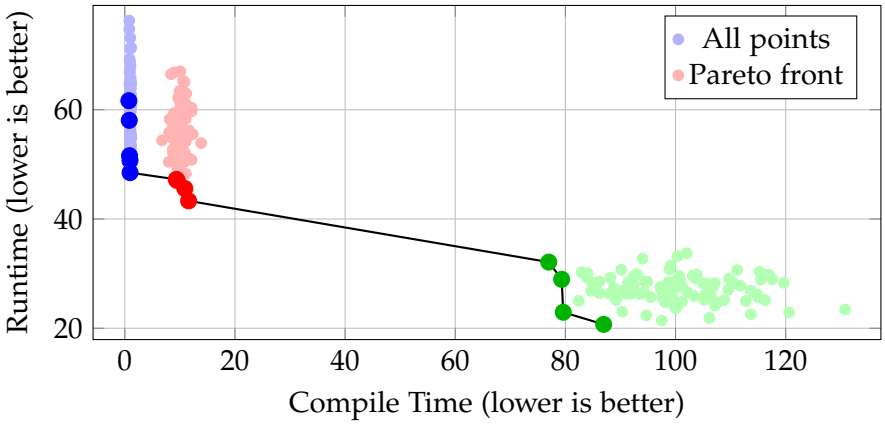
Contents

1 Introduction

- Compile speed is important for some applications like Database systems
- For short running programs, compile time is the dominant factor
- Gains from optimizations are not sufficient to amortize the additional compile time
- For long running programs, runtime is the dominant factor and more expensive optimizations are beneficial
- For most scenarios where a short compile time is wanted, the most interesting metric is the combined compile and runtime of a program citation
- TPDE [SKE25] is currently the fastest Compiler for LLVM-IR in unoptimized scenarios (-O0) targeting x84-64 and AArch64
- However, transitioning from TPDE to the next optimization level, LLVM [**llvm**] O1 pipeline is a large step in compile time
- compile time increases by approximately 10 times while delivering only around 5 times runtime improvement (as illustrated in Fig. **??**)
- Therefore there is a significant subset of programs in JIT scenarios that are likely to benefit from some level of optimization, but currently lack a fast enough solution

Approach

- fast accurate liveness analysis cite paper
- utilize SSA form to seperate spilling and register allocation phases
- Implememnt a fast spilling phase without requiring changes to the icnoming IR.
- spilling analysis without requiring prior isntruction selection phase.
- build a repairing register allocator to adress architecture constrints.



Contribution

- Building a simplified spilling mechanism whichi guarantees correctness (even for incorrect liveness data)
- spilling wihtout modifications to IR
- Improving Pareto optimal compilation trade-offs.

Outline

- Background
- Liveness
- Spilling
- Register allocation
- Related work
- Evaluation
- Conclusion

2 Background

In this chapter, we will introduce the basic concepts of register allocation and the particular importance of the SSA form. We then briefly describe the TPDE backend framework and the resulting challenges building register allocation on top of it.

2.1 Register Allocation on non-SSA programs

Register Allocation is the process of assigning values from a program to a limited number of k physical registers. This process is commonly modeled as finding the Graph Coloring of an Interference graph corresponding to the program.

The nodes in an interference graph correspond to the values in a program, and edges between them indicate that the connected values interfere with each other and therefore cannot be in the same register. This nicely corresponds to Graph coloring where connected nodes may not be assigned the same color.

Therefore, a valid coloring with at most k colors would be a valid register allocation for the corresponding program on a perfectly regular architecture with at least k registers.

In practice, using such an approach directly is unfeasible as GC is NP-complete in the number of values [GJ90], and any arbitrary IG can be used to construct a program having the same IG [Cha+81]. In addition, deciding how many registers a program requires and deciding whether k registers are sufficient are also NP-complete [Hac07].

Practical RA needs to rely on heuristics at all steps, to find allocations in a reasonable time. We will illustrate this with the seminal work by Chaitin [Cha+81] on the subject. Note that we will assume a theoretical regular architecture for now. We will address architectural constraints and their associated complications later.

2.1.1 Chaitin Style Graph Coloring Register Allocation

First the Interference Graph itself has to be constructed. This is done by scanning the program and adding edges between values that are live at the same time.

Next, a coalescing heuristic is applied such that the nodes stemming from copy instructions are merged into a single node. Notice that this step might increase the

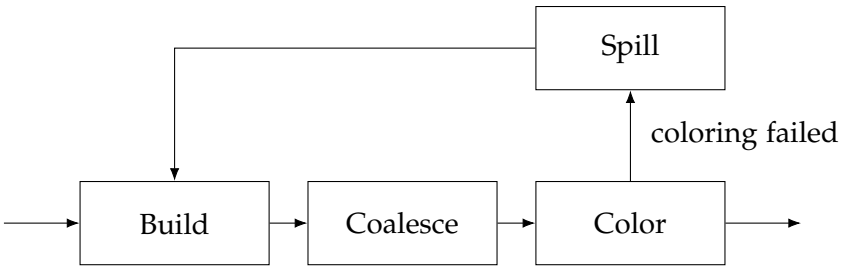


Figure 2.1: Simplified overview of the steps in a graph coloring allocator

number of colors required for the new graph since, the merged nodes may have a higher degree than the original nodes.

Finally, the main coloring step begins. First all nodes with fewer than k neighbors are removed from the graph and placed into a stack. If the graph is now empty, we move on to the next step. Otherwise, we select a remaining node for spilling and rebuild the IG for the new program after inserting spill code. Spilling removes edges from the graph, as the lifetime of the spilled value is reduced. This process is repeated until the graph is empty.

Now all nodes are on the coloring stack, where we reinsert them into the graph and assign them a color. This will always be possible since all nodes on the stack must have fewer than k neighbors. This produces a valid coloring of the final program.

While this approach produces good results in practice, building the Inference Graph is an expensive operation and the iterative nature of the algorithm make it unattractive for use in a JIT compiler [CD06].

While there have been multiple improvements to the original algorithm, the basic iterative structure remains the same, prompting different approaches to the problem.

2.1.2 Other Approaches

One alternate approach is to formulate Register Allocation as an Integer Linear Programming problem. While this can produce optimal results it is also NP-complete and the worst case runtimes of several minutes makes them unattractive for JIT compilers[FW02].

The other common approach is to use a linear scan allocator [PEK97]. This approach avoids the expensive IG data structure in favor of using a total ordering of all program instructions. This ordering is used to determine the single live range of each value, for example starting at instruction i and ending at instruction j ($[i, j]$). The live ranges are then assigned to registers in the order of their interval start.

Once the algorithm runs out of registers, it spills a value, splitting its live range into

```
%a   = ...
%b   = ...
%c   = ...
%d   = add i32 %b, %c
%e   = add i32 %d, %a
%a_1 = i32 %e (redefinition of a)
```

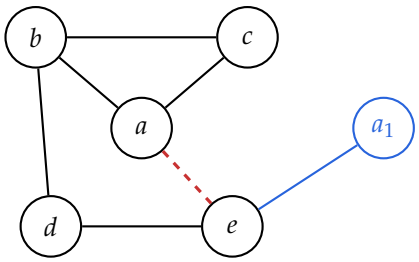


Figure 2.2: Example snippet of LLVM-IR. Assume that `%a_1` is a redefinition of `%a` in the original program. This would add the red edge in the interference graph, creating a cycle of length 4 with other edges connecting the cycle nodes. Therefore, the graph with the red edge is not chordal. The SSA version adds a new node `%a_1` that removes the cycle, therefore the graph with the blue elements remains chordal.

two parts. The spilled value is typically chosen based upon the furthest live range end.

This approach is simple and non-iterative and therefore attractive for use in a JIT compiler. However, due to the simplified liveness information, the resulting allocation quality is comparatively worse. Especially around control flow such as loops and branches with differing register pressure. Although there have been multiple improvements from the original algorithm, the resulting allocation quality is considered worse than that of graph coloring [WM05].

2.2 Register Allocation on SSA programs

Limitations of previously discussed approaches primarily stemmed from the NP-completeness of the underlying allocation problem. For SSA programs, the interference graph can no longer be arbitrary, but is always chordal [HGG06].

Chordal graphs are graphs where every cycle of length four or more has a chord, an edge connecting two vertices in the cycle that is not itself part of the cycle (see Fig. 2.2).

Chordal graphs have a number of useful properties, including the fact that they can be colored in polynomial time. Additionally, the required number of colors is equal to the size of the largest subset of vertices that are all adjacent in the graph, which is are determined by the program points with the maximal register pressure [Hac07].

SSA register allocators can therefore first use a spilling phase to reduce the register pressure to at most k at each program point, and then color the resulting chordal graph (this is possible even without constructing the graph explicitly [Hac07]).

Note that these results still assume a regular architecture. For resolving PHI nodes with cycles and loops, as well as constrained instructions, additional registers may

Figure 2.3: Simplified overview of the steps in an SSA register allocator.

be necessary. By splitting live ranges around constrained instructions some of these problems can be avoided, but doing so introduces additional work in a later coalescing phase [Hac07].

As our work is focused on register allocation speed rather than perfect allocation quality, and we lack some required information to employ these techniques, we do not consider these issues further.

2.3 TPDE

TPDE [SKE25] is a compiler backend framework focusing on short compilation time. It can be used with most SSA intermediate representations without a translation step, although this work will focus on LLVM-IR in particular. This also entails that TPDE cannot modify the IR in any way.

In TPDE functions consist of a sequence of basic blocks, which in turn consist of a sequence of instructions. Instructions are represented as a list of operands and a list of results both of which are made of values. Values can have multiple parts, where each part has a bank and size associated. Each part can be stored in their designated stack slot or in a register at any program point.

Lifetime is managed at a value level and is calculated using approximated liveness with a start and end basic block as well as a number of references, after which the Value is dead, and its stack slot can be reused.

TPDE combines instruction selection, register allocation and instruction encoding into a single code generation step. This is done by user provided instruction compilers for each IR instruction that encode the semantics of the instruction and compile them directly into machine instructions.

2.3.1 Instruction Compilers

Instruction compilers are responsible for getting handles to the actual Values, which prevent them from being spilled as long as they are alive. They then load the operands into registers or use them as memory operands and encode the instruction. In addition to the normal Values, they can also allocate scratch registers, which cannot be spilled

```
void emit_udiv64(IRInstr inst) {
    ValuePartRef lhs_ref = val_ref(inst->getOperand(0), 0); // Handle for operand 0, part 0
    ValuePartRef rhs_ref = val_ref(inst->getOperand(1), 0); // Handle for operand 1, part 0
    ScratchReg rdx_scratch{}; // Scratch for RDX, unspillable
    lhs_ref.into_specific(Reg::AX); // Move or load lhs into RAX
    ValuePartRef res_ref = result_ref_will_overwrite(inst, 0, std::move(lhs_ref));

    AsmReg rdx_reg = rdx_scratch.allocate_specific(Reg::DX); // reserve RDX for the Scratch
    ASM(XORrr, rdx_reg, rdx_reg); // Ensure RDX is zero
    AsmReg rhs_reg = val_as_reg(rhs_ref); // Force into arbitrary GP register
    ASM(DIV64rr, rhs_reg); // Encode instruction, clobbers first operand
    set_value(res_ref, res_ref.cur_reg()); // Notify framework that register was modified
} // Release handles and scratch
```

Listing 2.1: Example instruction compiler for an 64-bit unsigned division instruction for x86-64. The framework will move values into registers and create a copy of the first operand if required. We use a ScratchReg to clear the `RDX` register.

and are used in situations where a specific instruction requires an additional registers or clobbers other registers than the operands and results (see Lst. 2.1).

Instruction compilers can allocate more or less registers than is suggested by the number of their operands and results. As TPDE is unaware of the true semantics of the instruction, this makes register allocation, which has to operate before code generation, more difficult.

2.3.2 Current Register Allocation in TPDE

Whenever a register is requested by an instruction compiler, the framework will try to use the existing register if the requested part currently is in one, otherwise it will look for a free register from the correct bank. If no register is available, it will spill a register according to heuristic taking into account the cost of reloads as well as the remaining lifetime of the spilled value. For requests of a specific register, the value contained in that register will be spilled.

In addition to this basic mechanism, values that are used across basic blocks in the innermost loop can be assigned a fixed register. Such a fixed part will be in the same register for its entire lifetime and will never be spilled. This is targeted at loops to avoid spilling loop induction values or other values used in the innermost loop.

Otherwise, all non-fixed registers are spilled at each basic block boundary, unless the next block is the immediate successor and has only one predecessor. This avoids having to store the register state across basic blocks.

2.3.3 TPDE specific challenges for Register Allocation

As previously established, instruction compilers may require more or fewer register than is suggested by their operands and results. This makes register allocation perfect register allocation impossible, as the code generation step is final and intelligent register choices have to be calculated beforehand.

Fortunately in practice most instructions behave regularly. To this end, we will make the following regularity assumptions, which we assume to hold for most instructions and programs:

- Most Values have one part.
- Most instructions need exactly as many registers as their operands and results suggest.
- Most instructions do not need specific registers or clobber non-result registers.
- Most loops are executed often so that spills and reloads outside a loop are always preferred.
- Most loops are reducible.

Our work will utilize these assumptions while making sure the approach remains correct, but potentially suboptimal in the presence of irregularity.

3 Related Work

As explained previously in Section 2.3, our work has access to less information than is typical, therefore all approaches discussed in this chapter would require significant adaptations to be applicable in TPDE.

3.1 Graph coloring Register Allocation

As previously described, graph coloring of interference graphs is equivalent to register allocation [Cha+81]. In general this is an NP hard problem [Cha+81]. For graphs that are not colorable with k colors, the graph coloring algorithms additionally nedd to iterate between coloring, spilling and coalescing. Where each spill step requires an expensive rebuild of the interference graph[CD06]. This makes traditional graph coloring register allocation too slow for just in time compilation [CD06]. There are multiple improvements proposed to the original algorithm aiming to improve the code quality, such as optimistic coalescing [PM04].

The cost of rebuilding the interference graph after every spilling step can be avoided by approximating the resulting interference graph using additional metadata [CD06]. This sacrifices a small amount of code quality for a significant speedup, which makes it more suitable for just in time compilation. Still compared to other methods, the iterative nature of graph coloring is not desirable when prioritizing compilation speed. We leverage the SSA properties to do spilling as a preprocessing step, and avoid building the interference graph entirely.

We will consider such SSA-based algorithms such as ours separately as they have different compilation speed properties, even though they are fundamentally based on graph coloring.

3.2 Linear Scan Register Allocation

Linear Scan Allocators fundamentally of a single scan over the program, finding live ranges and then mapping these live ranges to registers (or memory) [PEK97]. Compared to Graph Coloring, Linear scan require only one scan over the program and no expensive interference graph construction. However, their code quality is generally

worse than graph coloring [Hac07]. While some limitations of the original algorithm, such as the lack of lifetime holes for control flow, have been addressed[WF10]. Linear Scan Allocators fundamentally use a non-optimal approach with increasingly elaborate heuristics or post-processing passes to improve code quality[WF10].

Similar to improvements by Wimmer and Mössenböck[WF10], our approach precomputes a set of spilled variables, allowing for better spill locations compared to spilling once the algorithm runs out of registers. Similarly we emphasize loops placing spills and reloads outside of loops as much as possible. As opposed to the later modifications to Linear Scan, we cannot change allocated code afterwards, which limits the effectiveness of some proposed improvements. In particular, moving spills would not be possible for our work.

3.3 Trace Register Allocation

For JIT Compilers in particular, trace allocation can be used, which assigns registers on chosen segments without control flow, referred to as traces [Eis15]. This allows for fast algorithm and can use different allocation strategies based on the trace properties. However, choosing good traces is non-trivial without runtime profiling information, and the resulting code quality is highly dependent on the trace selection.

As TPDE [SKE25] does not use have access to runtime profiling information, this approach is not suitable for our use case. Our approach allocates across control flow, without runtime information under the assumption that loops will be executed multiple times.

3.4 SSA Register Allocation

By utilizing the favorable properties of SSA form, these strategies can avoid the runtime costs of traditional graph coloring approaches. Although adapting these results for real architectures with constrained instructions and no performant swap instructions requires live range splitting and additional modifications to the IR[Hac07]. Such modifications would not be difficult to integrate into TPDE, which cannot modify the underlying IR.

Instead of the previously necessary modifications to the program, Colombet et al. [Col+11] introduce the notion of repairing if an irregularity is encountered. By allowing the register set to deviate at such points, they present a simpler algorithm that falls back to graph coloring in situations where a repairing using parallel moves along is not possible. We adopt a simplified notion of repairing, but are unable to use a fallback to graph coloring, as code generation cannot be rolled back.

4 Approach

This chapter describes how we extend the TPDE framework to support better register allocation choices, while keeping the core code generation as a single pass.

4.1 Liveness with next-use Distance

As described in Section 2.3, the existing liveness analysis in TPDE is on the Basic Block level and does not support lifetime holes.

This is insufficient for a effective register allocation, as it creates too many interference between variables, that are not actually present in the original program.

To resolve this we introduce a precise liveness analysis, that also calculates the next-use distance at each point in the program.

For this we adapt the approach by Boissinot, Brandner, Darte, et al. [Boi+11].

We define the next-use as the number of instructions between *start of the current block* and the next use of the variable. This means that next-uses are always strictly increasing in a block. The next-use distance can be easily calculated using the index of the current instruction and the next-use.

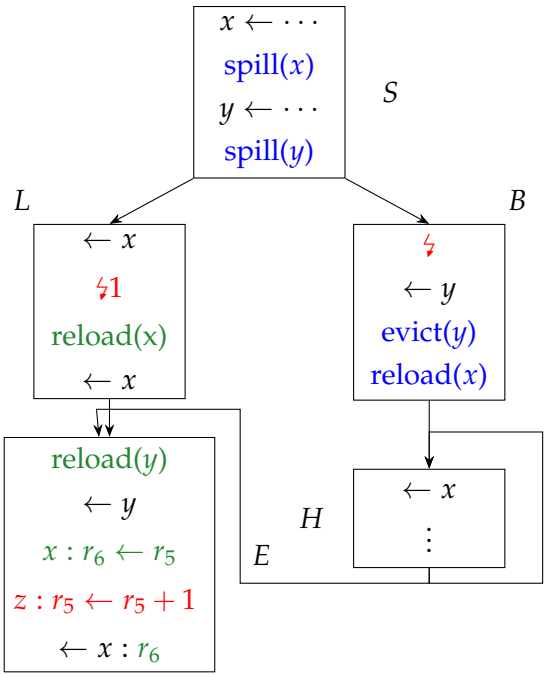
4.1.1 Datastructures

We chose to store this information as a sorted (by construction) list of next-uses for each variable and block. Definitions are marked by having their highest bit set, therefore a value is defined in a block if its first next-use entry has the highest bit set (and is not dead). Dead values have a next-use of `u32::max` or no entry of the value is neither live-in nor defined in the block. A value is therefore live-out if its last entry is not `u32::max`.

For example, the following array describes a value that is defined in the second instruction of the block, then used in the third (as an operand) and is dead afterwards.

0	1	2
`(1«31) | 1`	3	`u32::max`

If a values is live-out, its last entry would be the size of the block plus the minimum next-use of the successor blocks, ensuring that the list remains sorted.



1. Lifetime analysis with calculation of next-use distances
2.1. Spilling pass to reduce register pressure at critical points (↯)
2.2. During spilling calculate optimal loop header register state
3. Codegen with repairing at constrained instructions (see z) and insertion of reloads.

In addition, we apply a loop exit penalty for the live-out distances if the sucessor is outside the current loop. This is simply a large number added to the distance, that accounts for loops being executed often, according to our regularity assumtions (see Section 2.3). This will encourage the later spilling pass to avoid spilling loop variables [BH09].

Overall this allows for finding the next-use of a values in logarithmic time in the number of uses in a basic block. As well as checking live-in, live-out and wether it was defined in the block in constant time. In addition checking a value is used in a loop is also constant time, as we assume all other uses to be below the chosen loop exit penalty.

4.1.2 Partial Liveness

First we calculate partial liveness for each block by traversing the CFG in post order, skipping loop edges. For each block we precess the instructions in forward order, while tracking the current instruction index and noting the definitions and uses of each variable.

To find the live out distances, we iterate over the direct suceessors, once again skipping loop edges and taking the minimum next-use from the sucessors. As we are traversing the CFG in post order, the non-loop successor blocks are already processed.

Note that we consider values that are used only in a phi definition not live-in, but

we will consider them live-out at their predecessor blocks with a use at the zeroth instruction of the successor.

4.1.3 Liveness propagation

To propagate the liveness information through the program, we calculate a loop forest as described by [Boi+11].

We propagate liveness, by marking live-in values from a loop header as live thoughout all child loops (the children of the loop header in the loop forest). This corresponds to adding a single next-use entry for each value in the children of the loop header. The next-use entry is larger than the size of block indicating the value is live-in and live-out, but neither used nor defined in the block.

Notice that this calculation is not correct for irreducible loops, that have multiple loop headers. As we consider irreducible loops to be rare, we do not handle this case as the insufficient liveness information will not affect the correctness of the final generated program. For further details on how to adapt this liveness analysis for irreducible control flow, see [Boi+11].

4.2 Spilling

The primary purpose of the spilling pass is to ensure that the register pressure. To achviele this a spilling pass will typically insert spills and loads into the IR. For SSA form, this will also require inserting phi nodes as the reloaded value has to be distinct from the original value, since any value must have excately one definiton to remain in SSA form. Instead we only store the results of the spilling pass, and ensure that blocks are correctly connected, with each value in one unique location during codegneration.

4.2.1 Representing Spills and Reloads

Since we can't modify the existing IR, inserting the calculated spills and reloads is not possible. In particular we would need to check at every instruction wether there is a reload or spill that needs to be executed. While this would be possible, it requires a lot of book keeping. Instead we simplify the representations of spills and reloads. For reloads, we avoid storing them unless we are branching to a loop header. Otherwise the codegeneration will simply load the value once it is needed.

For spills we only store wether a value should be spilled according to the analysis. Then during codegeneration we insert the spill immediately after its definition. This simplifies analysis, as we don't need to track which vales where spilled in which

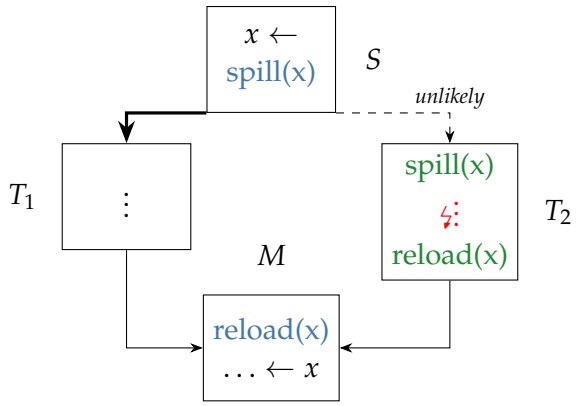


Figure 4.1: Example of a bad case for our simplified analysis. Green represents the ideal spill and reload locations. Blue is our approximation. T_2 is assumed to have high register pressure, therefore x has to be spilled. We assume that the branch to T_2 is unlikely, therefore we generate unnecessary spills and reloads.

branches, to avoid values being lost or unnecessarily spilled again. During codegeneration we only check a single bit in a bitset for the results, reducing the overhead of the spilling pass.

This approximation of spills does not result in the optimal solution in all cases, especially around blocks with very different execution frequencies (see Figure 4.1). This is particularly relevant for loops, which we assume to execute often.

To avoid a similar case as in Figure 4.1 for loops, we store the ideal register set for the loop header separately. During code generation the appropriate spills and reloads are generated to match this register set. This ensures that as few spills and reloads as possible are generated within the loop, while still avoiding storing additional state for most basic blocks.

4.2.2 Calculating spills

We use a simplified `MIN` algorithm based on the work of Braun and Hack.

For each block we simulate the register set W by limiting the size of W to k registers. For values that are live but need to be removed from W, we mark them as spilled. Values with multiple parts always need to be fully in W or fully spilled.

We spill according to the next-use distance, once before the instruction for operands and once after the instruction to ensure space for the results.

At this stage, we do not consider most instruction irregularities, except for calls.

Calls are treated with a number of implicit results equivalent to the number of volatile registers. All other irregularities are ignored (and are not known to TPDE at this stage).

Algorithm 2 Simplified `MIN` algorithm for a single basic block.

```
def LIMIT(W, insn, cap)
    sortByNextUse(W, insn)
    for v ∈ W[cap : −1] :
        if nextUse(v) ≠ ∞ :
            mark v as spilled
    W ← W[0 : cap]

def MINALGORITHM(block, W)
    for insn ∈ block.instr :
        W ← W ∪ insn.operands
        limit(W, insn, k)
        limit(W, insn.next, k − |insn.results|)
        W ← W ∪ {insn.results}
```

We start with W containing the function arguments. Then for each block other block we initialize W with the intersection of all incoming values from predecessors. Since we don't insert reloads at this stage, the initialization is not critical, unless the current block is a loop header.

For loop headers, we initialize W to only contain variables used in the loop and restrict it to $k - p$ registers, where p is the approximate maximum register pressure in the loop. This pressure p is calculated during the liveness analysis and is simply the maximum number of live variables in any of the loop blocks. This ideally ensures that the loop can be executed without spilling. Since our register pressure p is only an approximation, we conservatively evict all values that are not used in the loop.

5 Evaluation

5.1 Correctness

We verify the correctness of our implementation by using the LLVM test suite and by running the SPEC CPU2017 integer benchmarks. Except for some LLVM test cases that contain unsupported instructions, all tests pass sucessfully on AArch64 and x86-64. Which we assume is a good proxy for the correctness of our implementation.

5.2 Performance

5.2.1 Setup

We evaluate the performance of our implementation by measuring the back-end compile-time and the run-time performance of LLVM-IR generated by Clang. We use code produced with -O1, where variables are in SSA form. We compare our implementation against the original version of TPDE and against both the LLVM-O0 and LLVM-O1 back-end, the former two focusing on compile-time and the latter being a optimizing back-end. As benchmark programs, we use the SPEC CPU2017 integer benchmarks on x86-64 and AArch64. Our x86-64 machine is an Amd Ryzen 5 5600X with 32 GiB of RAM running Linux version; Our AArch64 machine is an Apple M1 with 16 GiB of memory, using only the four performance cores, running Asahi Linux version. We use Clang/LLVM version 20.1.8.

5.2.2 Optimized IR

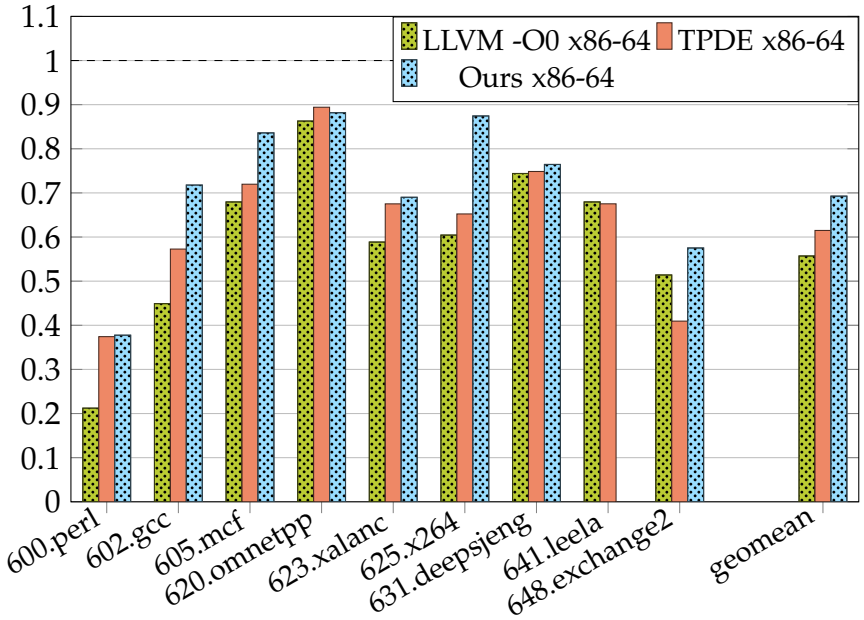


Figure 5.1: SPEC CPU2017 integer runtime speedup relative to clang-O1 with optimized IR.

6 Conclusion

6.1 Future Work

- Register Aliasing
- Irreducible Control flow
- rest was ich nicht geschafft hab.
-

Abbreviations

List of Figures

List of Tables

Bibliography

[BH09] M. Braun and S. Hack. "Register Spilling and Live-Range Splitting for SSA-Form Programs." In: *Compiler Construction*. Ed. by O. de Moor and M. I. Schwartzbach. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 174–189. ISBN: 978-3-642-00722-4.

[Boi+11] B. Boissinot, F. Brandner, A. Darte, B. D. de Dinechin, and F. Rastello. "A Non-iterative Data-Flow Algorithm for Computing Liveness Sets in Strict SSA Programs." In: *Programming Languages and Systems*. Ed. by H. Yang. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 137–154. ISBN: 978-3-642-25318-8.

[CD06] K. Cooper and A. Dasgupta. "Tailoring graph-coloring register allocation for runtime compilation." In: *International Symposium on Code Generation and Optimization (CGO'06)*. 2006, 11 pp.–49. DOI: 10.1109/CGO.2006.35.

[Cha+81] G. J. Chaitin, M. A. Auslander, A. K. Chandra, J. Cocke, M. E. Hopkins, and P. W. Markstein. "Register allocation via coloring." In: *Computer Languages* 6.1 (1981), pp. 47–57. ISSN: 0096-0551. DOI: https://doi.org/10.1016/0096-0551(81)90048-5.

[Col+11] Q. Colombet, B. Boissinot, P. Brisk, S. Hack, and F. Rastello. "Graph-coloring and treescan register allocation using repairing." In: *Proceedings of the 14th International Conference on Compilers, Architectures and Synthesis for Embedded Systems*. CASES '11. Taipei, Taiwan: Association for Computing Machinery, 2011, pp. 45–54. ISBN: 9781450307130. DOI: 10.1145/2038698.2038708.

[Eis15] J. Eisl. "Trace register allocation." In: *Companion Proceedings of the 2015 ACM SIGPLAN International Conference on Systems, Programming, Languages and Applications: Software for Humanity*. SPLASH Companion 2015. Pittsburgh, PA, USA: Association for Computing Machinery, 2015, pp. 21–23. ISBN: 9781450337229. DOI: 10.1145/2814189.2814199.

[FW02] C. Fu and K. Wilken. "A faster optimal register allocator." In: *35th Annual IEEE/ACM International Symposium on Microarchitecture, 2002. (MICRO-35). Proceedings.* 2002, pp. 245–256. DOI: 10.1109/MICRO.2002.1176254.

[GJ90] M. R. Garey and D. S. Johnson. *Computers and Intractability; A Guide to the Theory of NP-Completeness*. USA: W. H. Freeman & Co., 1990. ISBN: 0716710455.

[Hac07] S. Hack. "Register allocation for programs in SSA Form." PhD thesis. 2007. 123 pp. ISBN: 978-3-86644-180-4. DOI: 10.5445/KSP/1000007166.

[HGG06] S. Hack, D. Grund, and G. Goos. "Register Allocation for Programs in SSA-Form." In: *Compiler Construction*. Ed. by A. Mycroft and A. Zeller. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 247–262. ISBN: 978-3-540-33051-6.

[PEK97] M. Poletto, D. R. Engler, and M. F. Kaashoek. "tcc: a system for fast, flexible, and high-level dynamic code generation." In: *Proceedings of the ACM SIGPLAN 1997 Conference on Programming Language Design and Implementation*. PLDI '97. Las Vegas, Nevada, USA: Association for Computing Machinery, 1997, pp. 109–121. ISBN: 0897919076. DOI: 10.1145/258915.258926.

[PM04] J. Park and S.-M. Moon. "Optimistic register coalescing." In: *ACM Trans. Program. Lang. Syst.* 26.4 (July 2004), pp. 735–765. ISSN: 0164-0925. DOI: 10.1145/1011508.1011512.

[SKE25] T. Schwarz, T. Kamm, and A. Engelke. *TPDE: A Fast Adaptable Compiler Back-End Framework*. 2025. arXiv: 2505.22610 [cs.PL].

[WF10] C. Wimmer and M. Franz. "Linear scan register allocation on SSA form." In: *Proceedings of the 8th Annual IEEE/ACM International Symposium on Code Generation and Optimization*. CGO '10. Toronto, Ontario, Canada: Association for Computing Machinery, 2010, pp. 170–179. ISBN: 9781605586359. DOI: 10.1145/1772954.1772979.

[WM05] C. Wimmer and H. Mössenböck. "Optimized interval splitting in a linear scan register allocator." In: *Proceedings of the 1st ACM/USENIX International Conference on Virtual Execution Environments*. VEE '05. Chicago, IL, USA: Association for Computing Machinery, 2005, pp. 132–141. ISBN: 1595930477. DOI: 10.1145/1064979.1064998.